

CODE SPORT



In this month's column, we continue our discussion of neural networks and also go over a few computer science interview questions that focus on data science.

In my previous column, we started discussing neural networks, after which some of you have asked me to provide a list of resources or reading material on the topic. At the end of this column, I have provided a few pointers for learning more about neural networks. One question many readers had was about the differences between traditional machine learning and neural networks.

In traditional machine learning, much of the effort is focused on identifying the various features and then representing the input examples in terms of these features, as the input to the classifier. For instance, let us consider a simple classification problem. Given a sentence, we have been asked to classify the sentiment polarity associated with that sentence—whether it bears a positive sentiment, negative one or whether it is neutral, expressing no sentiment at all.

Let us assume that we are provided with 1000 sentences that form the training set for the classifier, and 50 test input sentences for which we need to predict the sentiment. Each of the input sentences need to be first represented in terms of their feature vectors. A typical set of features includes the *Bag of Words* feature, where the presence/absence of each word in the vocabulary is a binary feature and the sentence can be represented using the *Bag of Words* feature vector. Additional features can include the *Parts of Speech* tags for the constituent words of the sentence, the presence of sentiment carrying words in the sentence, the presence of negation in the sentence, etc.

Thus, each input sentence is transformed by software into a feature vector, which is then fed to the classifier model, be it Naïve Bayes or support vector machines. The training examples represented as a combination of the feature vector and target labels are used to train the classifier during the training phase. During the classification phase,

each test input sentence is converted into the feature vector, and then the classifier generates the predicated classification for each such sentence.

However, note that the task of identifying which features are to be a part of the representative feature vector is not automated but left to us humans to handle. Feature selection is arguably one of the more cumbersome and error-prone tasks in any ML problem. Each and every feature that can be useful in discriminating among the multiple categories needs to be identified and extracted before being fed as input to the classifier. This requires considerable knowledge of the problem and, in fact, without the right set of features, even the best classifier cannot perform well.

On the other hand, let us consider a neural network model for sentence classification. Instead of having to hand-craft different features, the neural networks just need to be supplied with core features. In fact, the input sentence represented as an n-gram can be presented as input to a convolutional neural network which can then classify the sentence. There has been considerable work in sentence classification using neural networks. An example of this can be found in the paper by Harvard PhD student, Yoon Kim, in which convolutional neural networks have been used for sentence classification. The code for this work is available at <https://github.com/yoonkim>.

As seen above, neural networks obviate the need for extensive hand-crafted feature engineering. However, one needs to understand that there is a trade-off involved here. Though not much of an effort is required for feature selection with neural networks, coming up with a suitable neural network architecture for each of the problems needs some hard work. For instance, there is considerable effort involved in deciding the type of neural network to use. Should we use a feed-forward, a recurrent or a convolutional